

HUFFMAN ENCODING

Encoding and Compression of Data

- Applications:
 - Fax Machines
 - Secret messages
 - Cryptography
- Advantages:
 - min number of bits needed
 - cost of savings

Building a Tree

Scan the original text

- Consider the following short text:

Eerie eyes seen near lake.

- Count up the occurrences of all characters in the text

Building a Tree

Scan the original text

Eerie eyes seen near lake.

- What characters are present?

E e r i space
y s n a r l k .

Building a Tree

Scan the original text

Eerie eyes seen near lake.

- What is the frequency of each character in the text?

Char	Freq.	Char	Freq.	Char	Freq.
E	1	y	1	k	1
e	8	s	2	.	1
r	2	n	2		
i	1	a	2		
space	4	l	1		

Building a Tree

Prioritize characters

- Create binary tree nodes with character and frequency of each character
- Place nodes in a priority queue
 - The lower the occurrence, the higher the priority in the queue
 - Similar to Min-Heap
 - modify code of min-heap as per this algorithm if required

Building a Tree

Prioritize characters

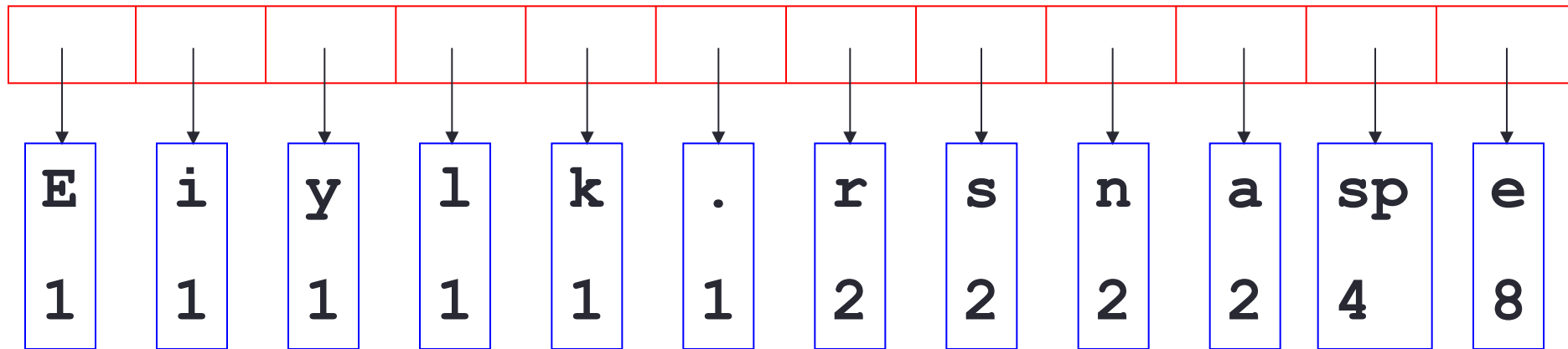
- Uses binary tree Nodes and Priority Queues (Min-Heap)

```
struct HuffNode
{
    char myChar;
    int myFrequency;
    HuffNode *left, *right;
};
```

```
minHeap myPriorityQueue;
```

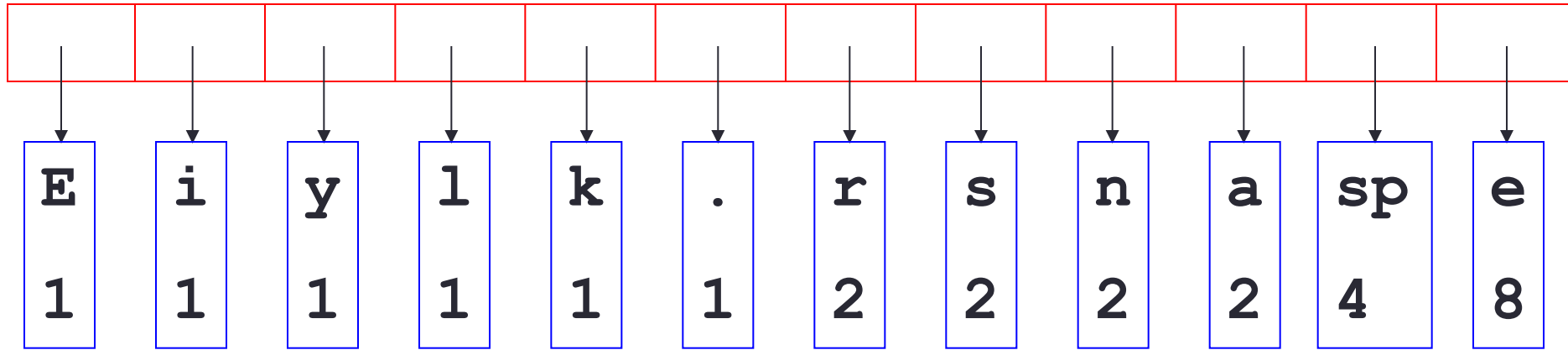
Building a Tree

- If frequency is same, insert characters based on order of occurrence in text
- The queue after inserting all nodes:

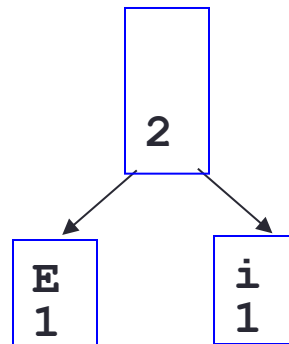
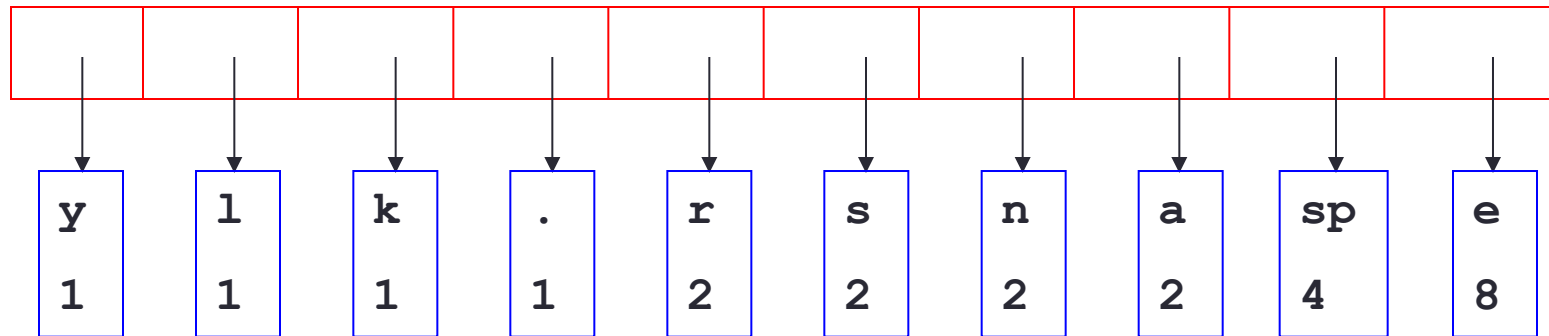


- Null Pointers are not shown

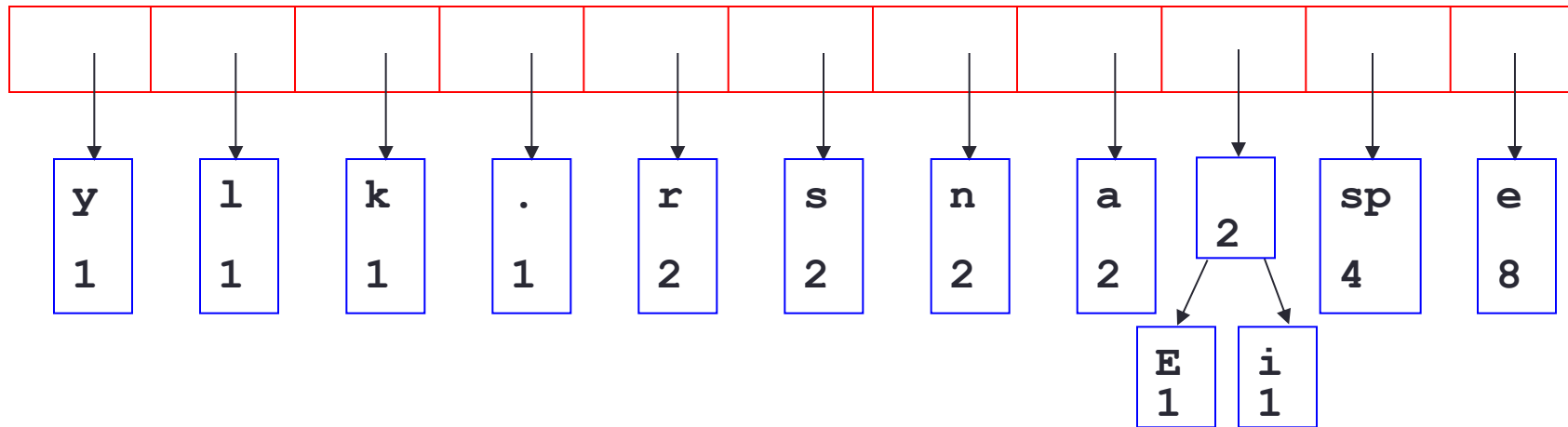
Building a Tree



Building a Tree



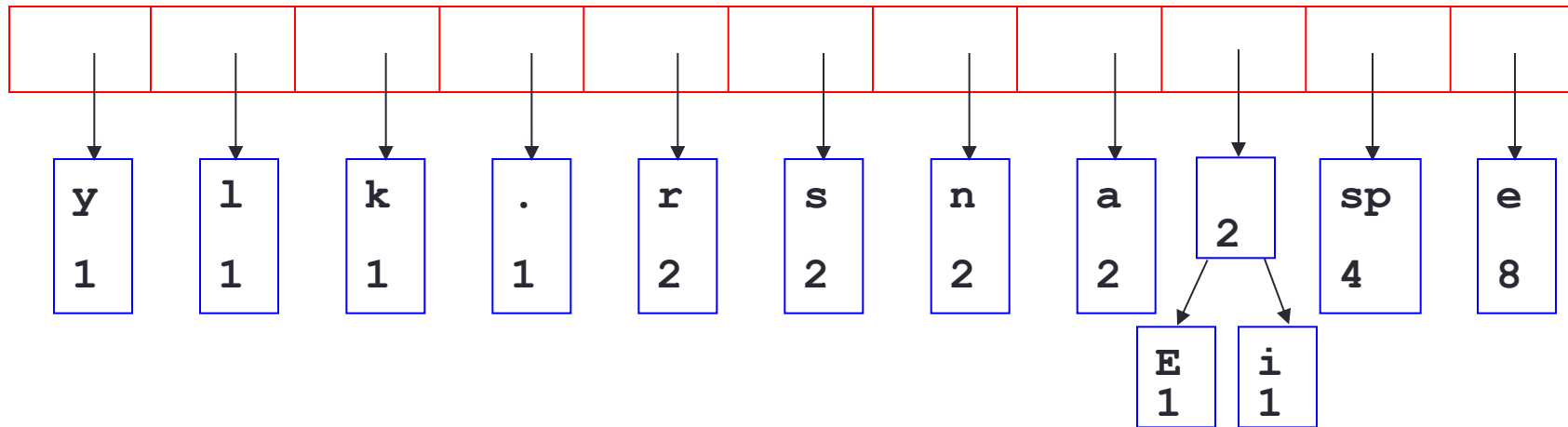
Building a Tree



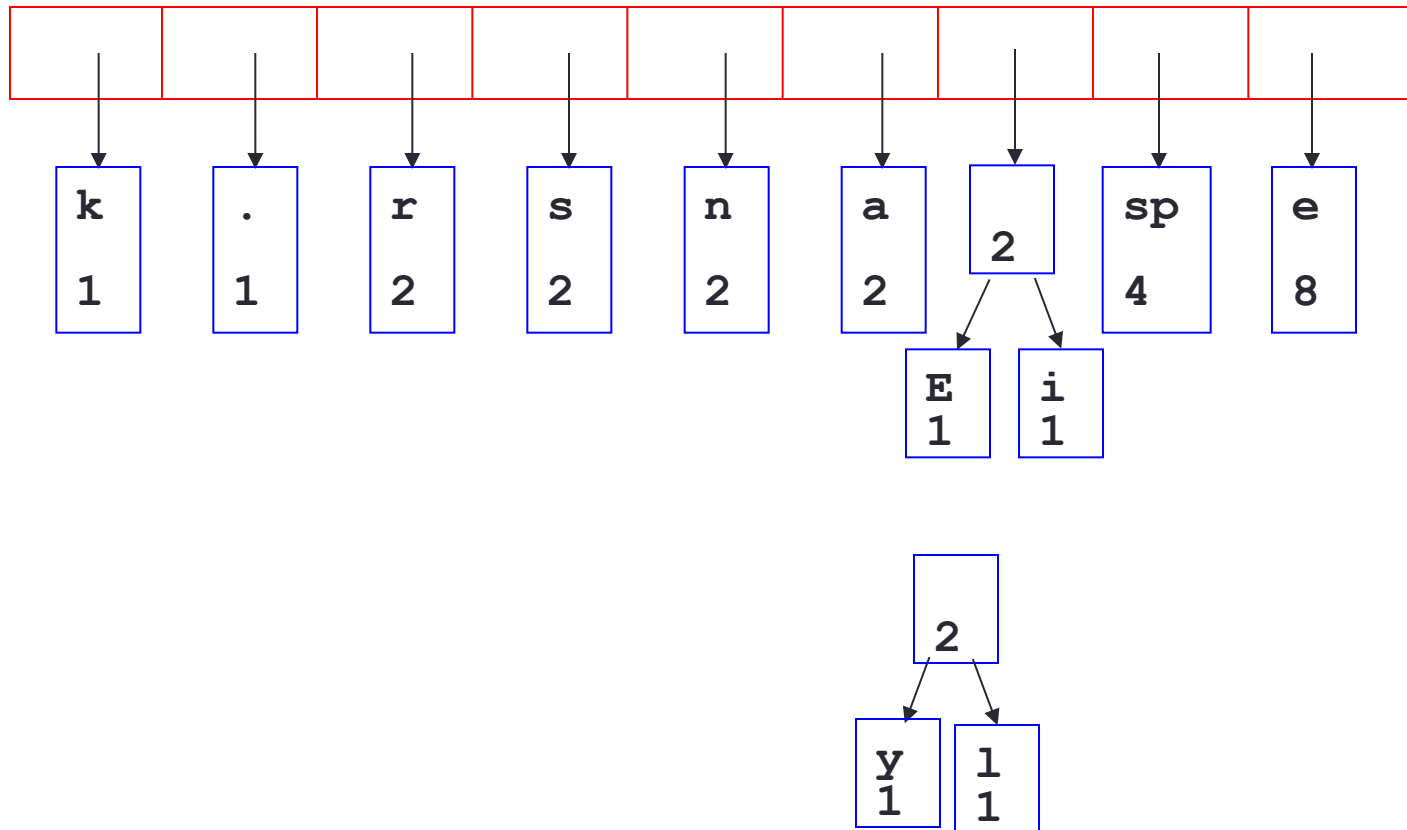
Building a Tree

- While priority queue contains two or more nodes
 - Create new node
 - Dequeue node and make it left subtree
 - Dequeue next node and make it right subtree
 - Frequency of new node equals sum of frequency of left and right children
 - Enqueue new node back into queue

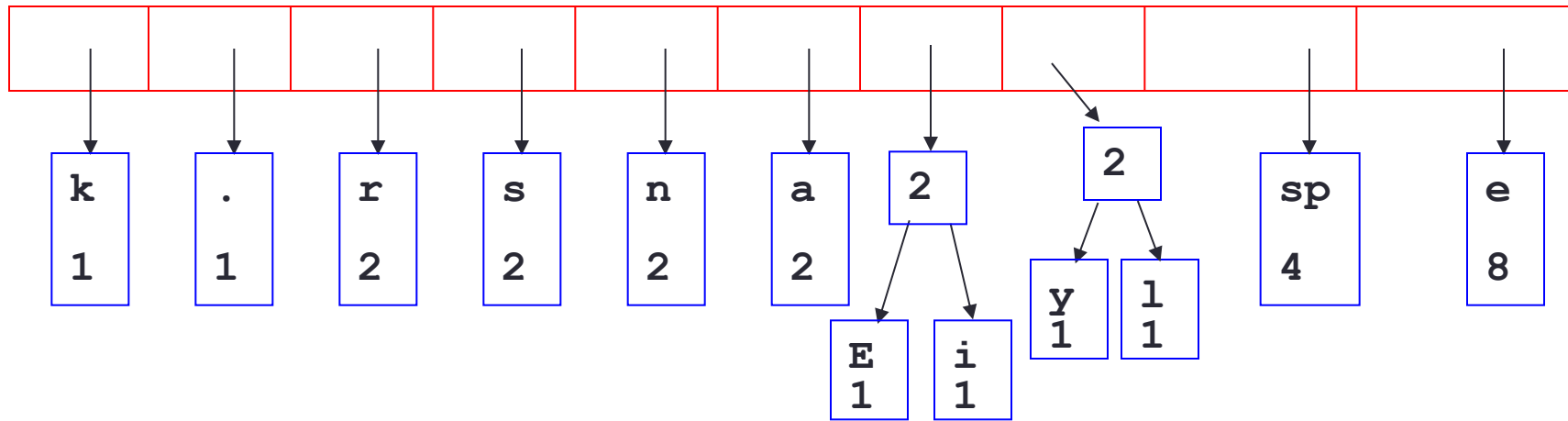
Building a Tree



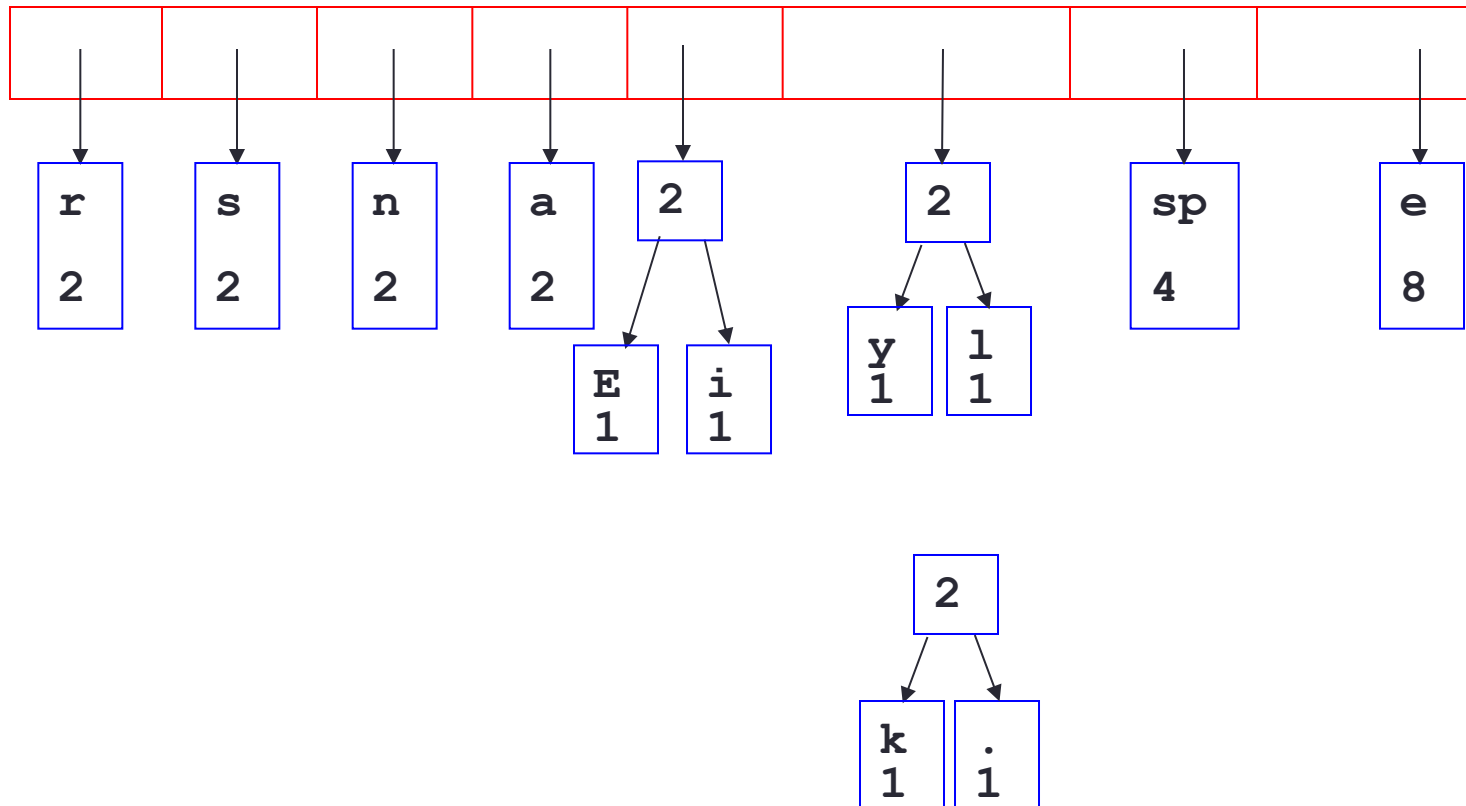
Building a Tree



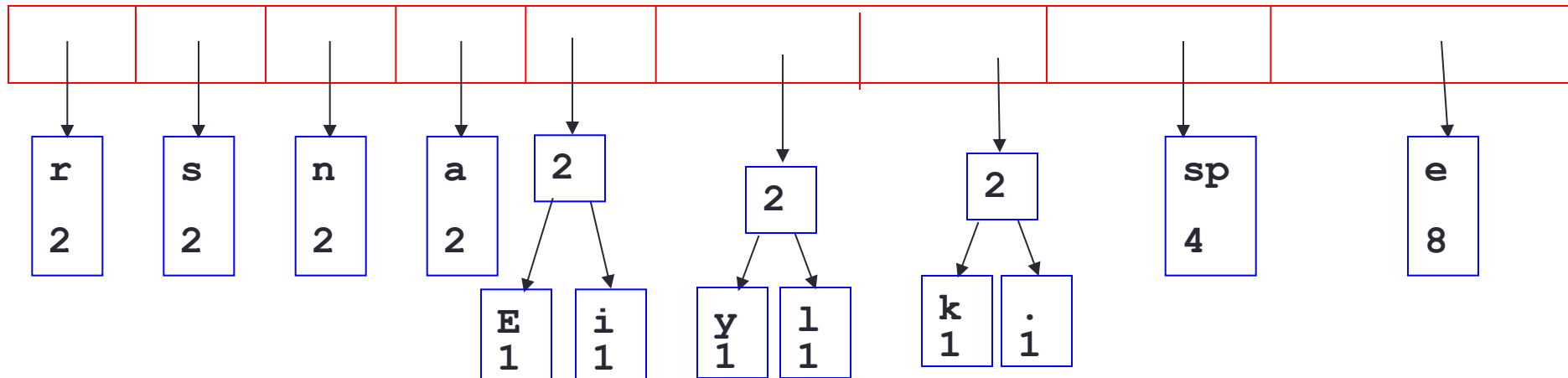
Building a Tree



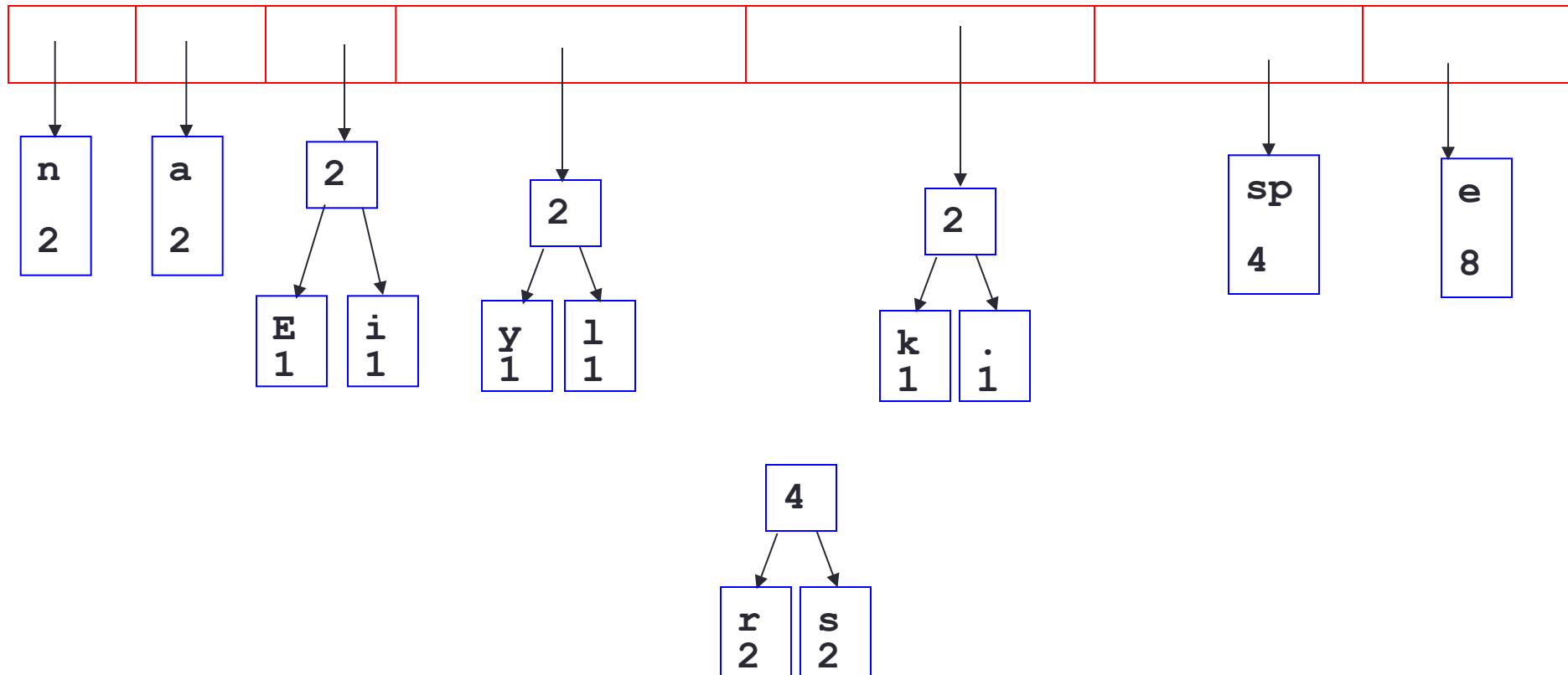
Building a Tree



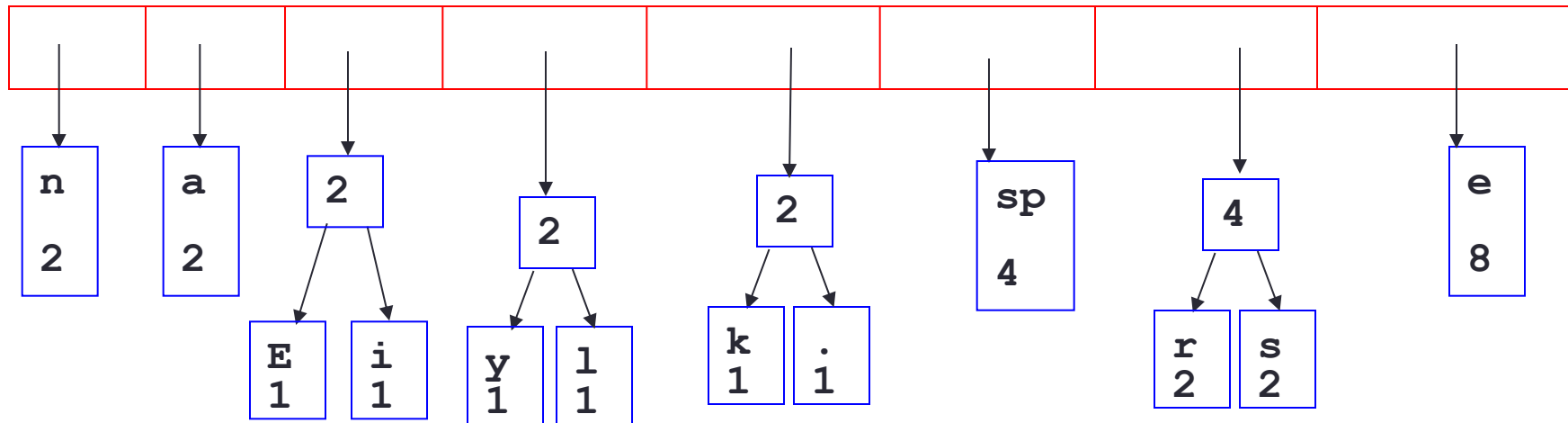
Building a Tree



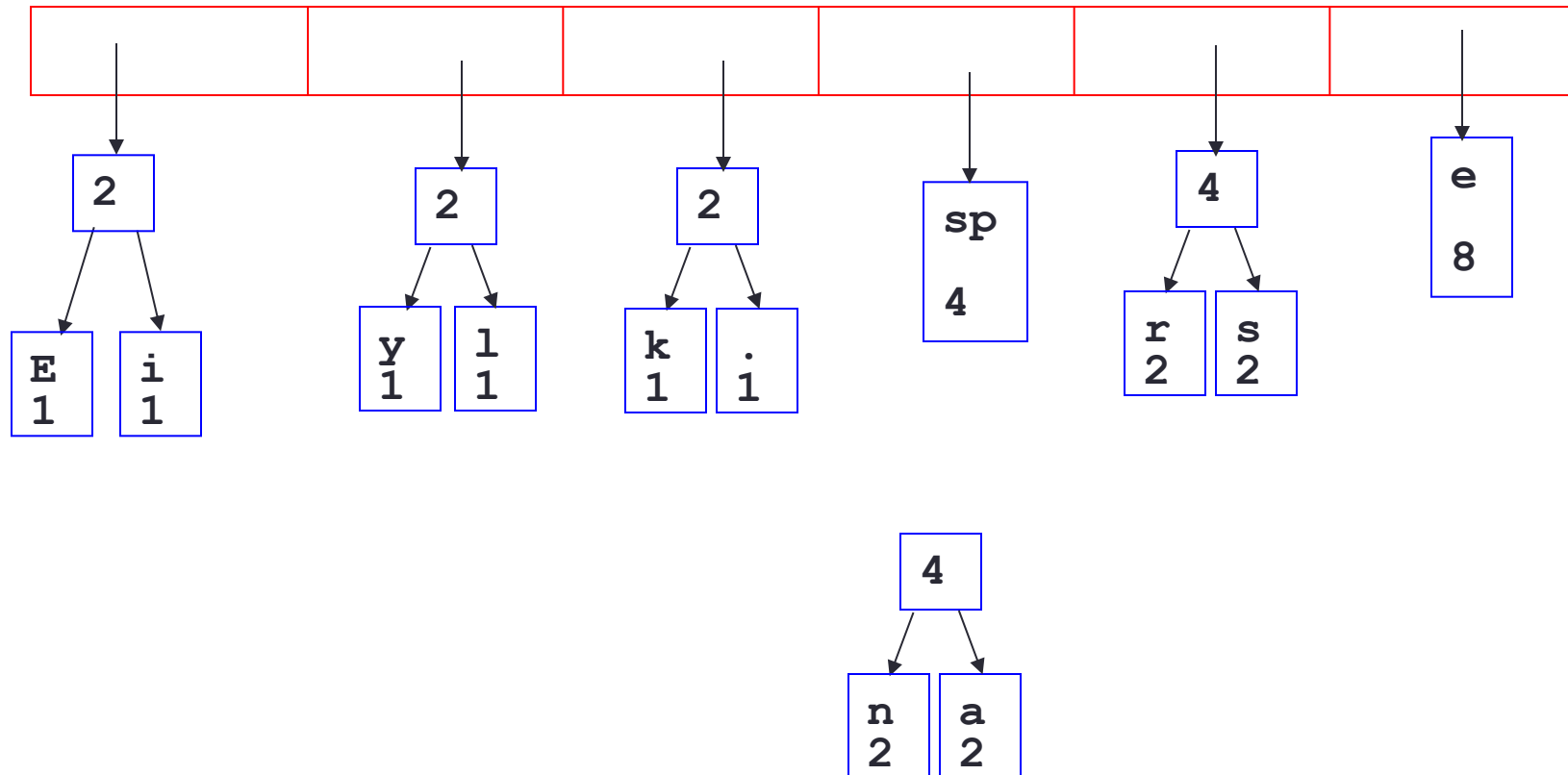
Building a Tree



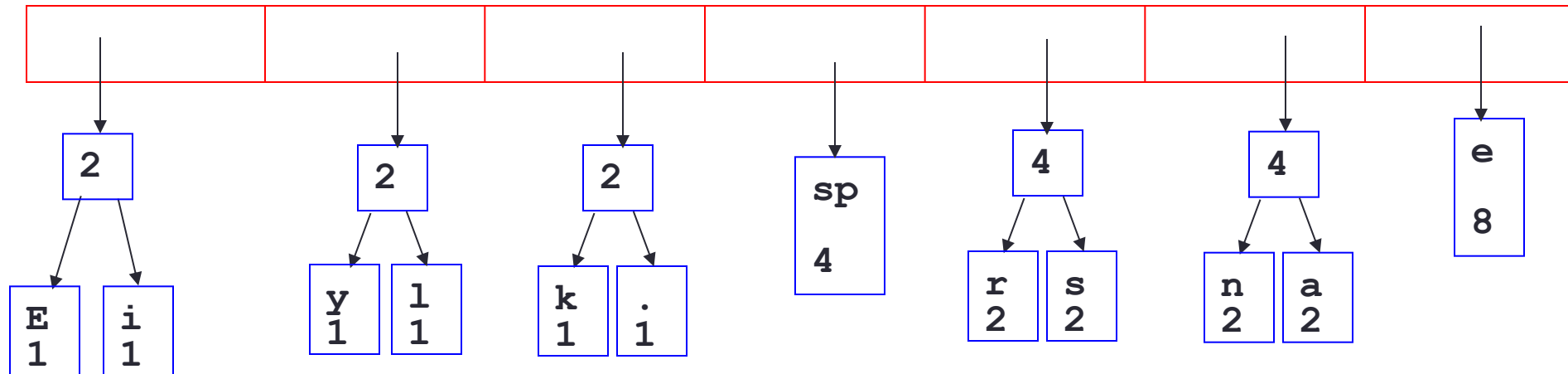
Building a Tree



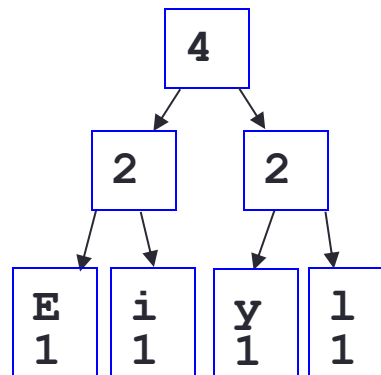
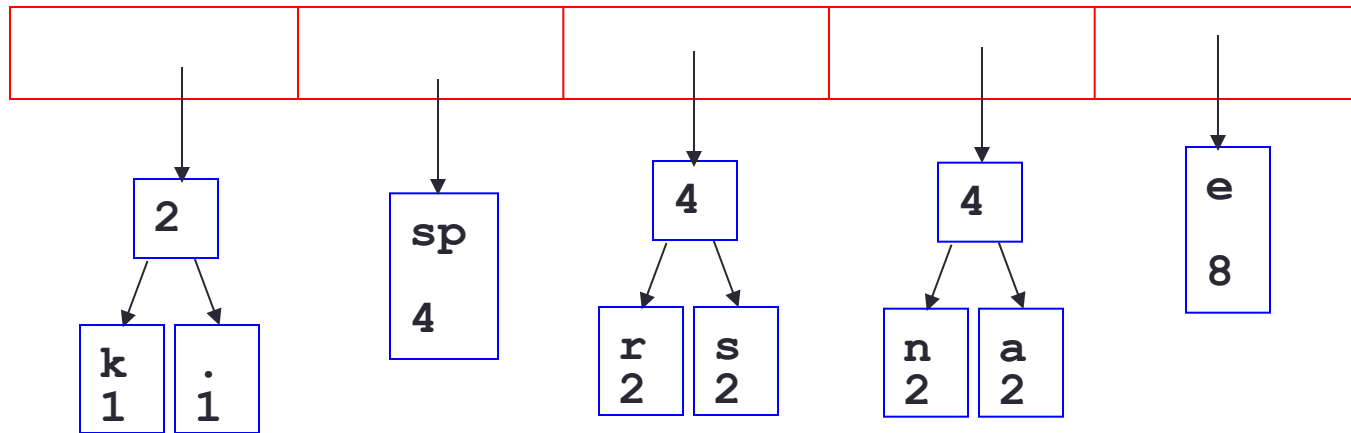
Building a Tree



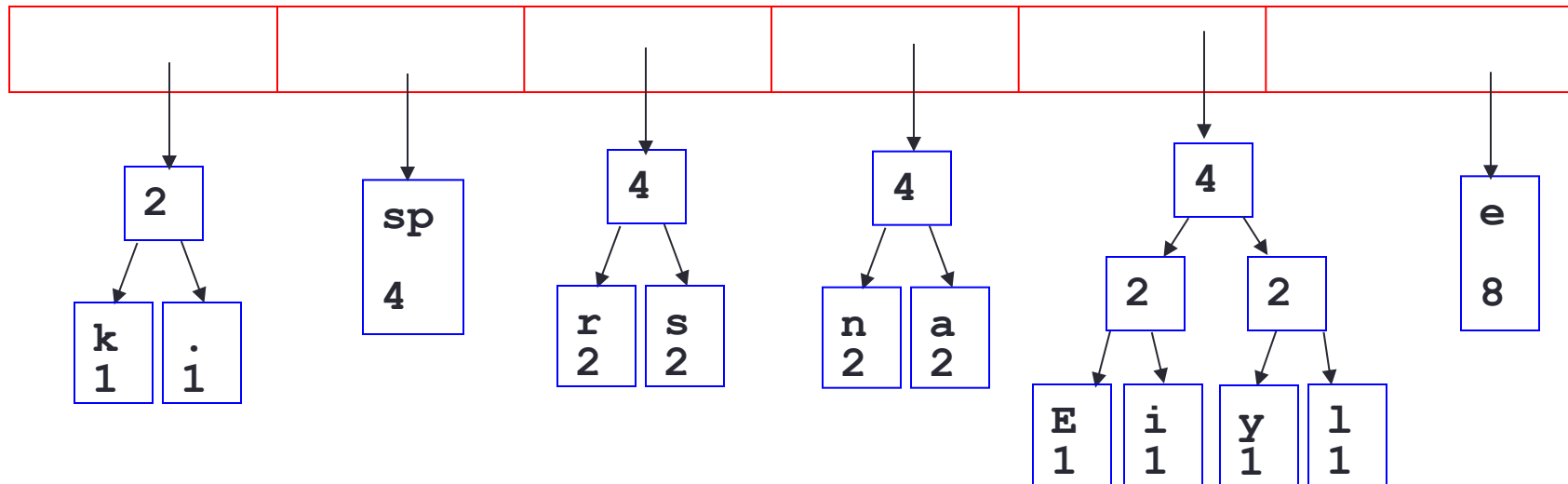
Building a Tree



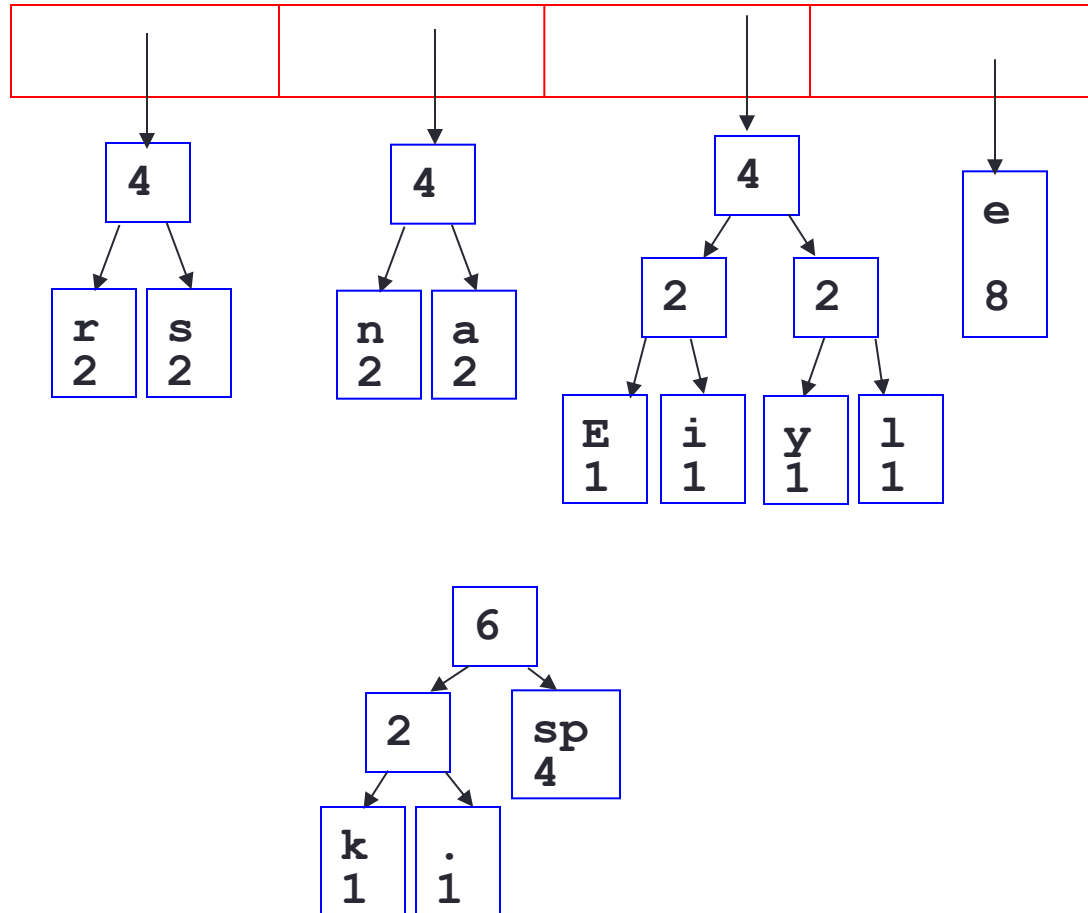
Building a Tree



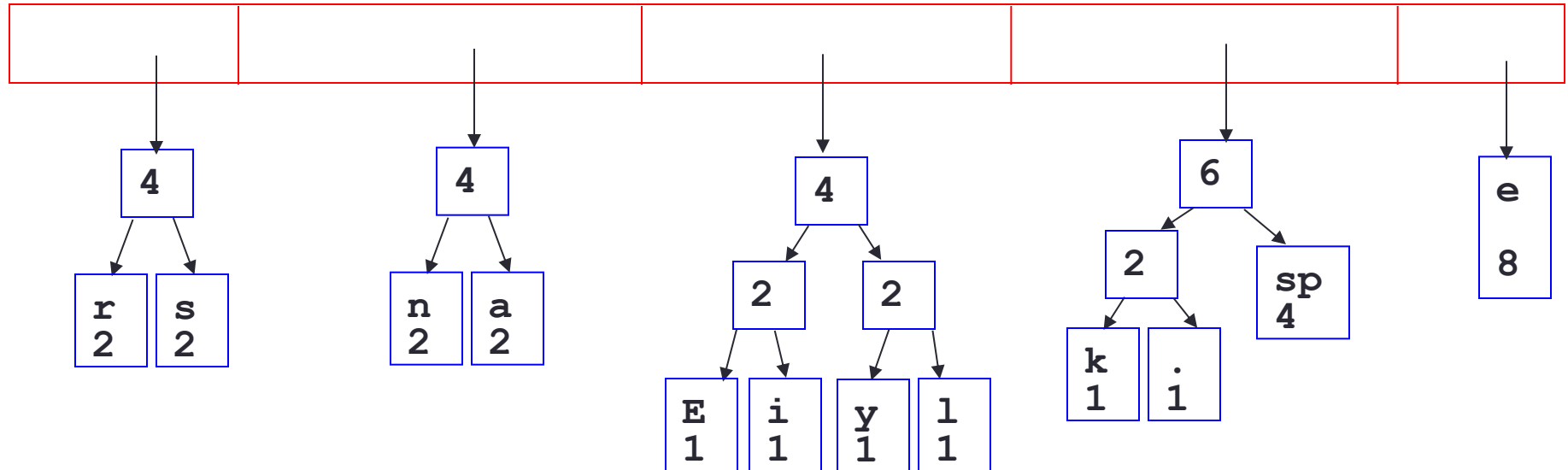
Building a Tree



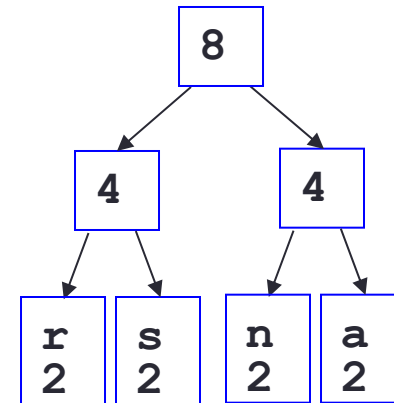
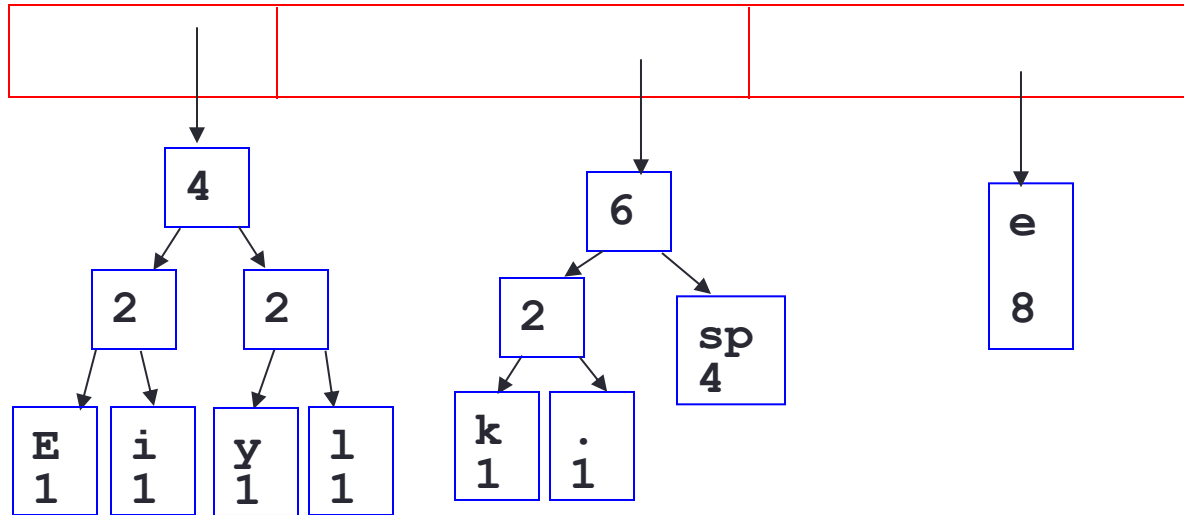
Building a Tree



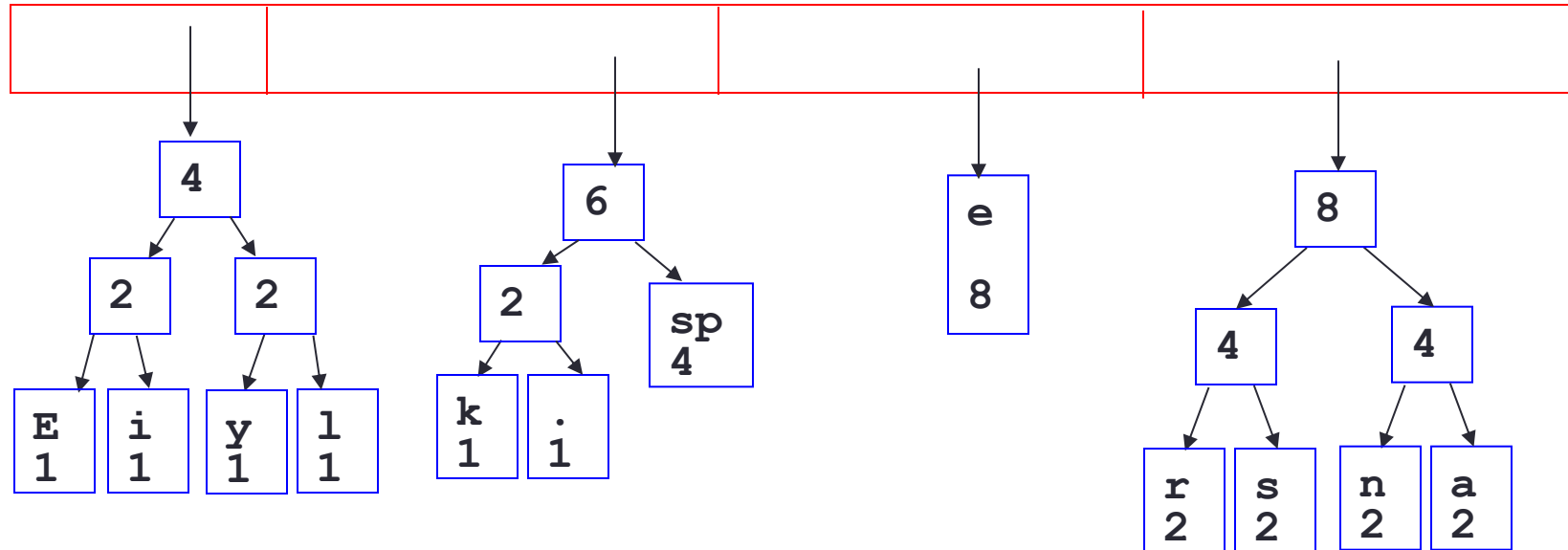
Building a Tree



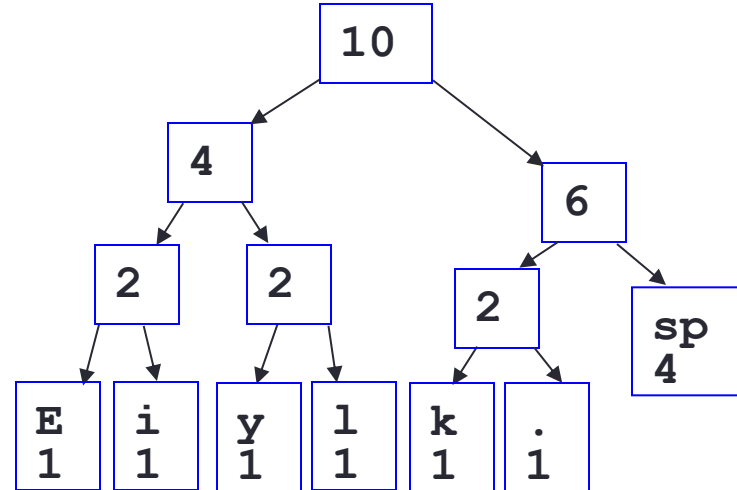
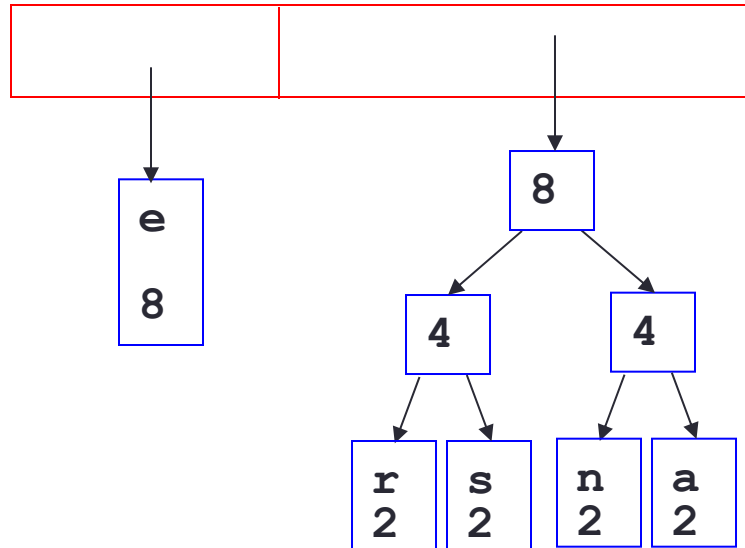
Building a Tree



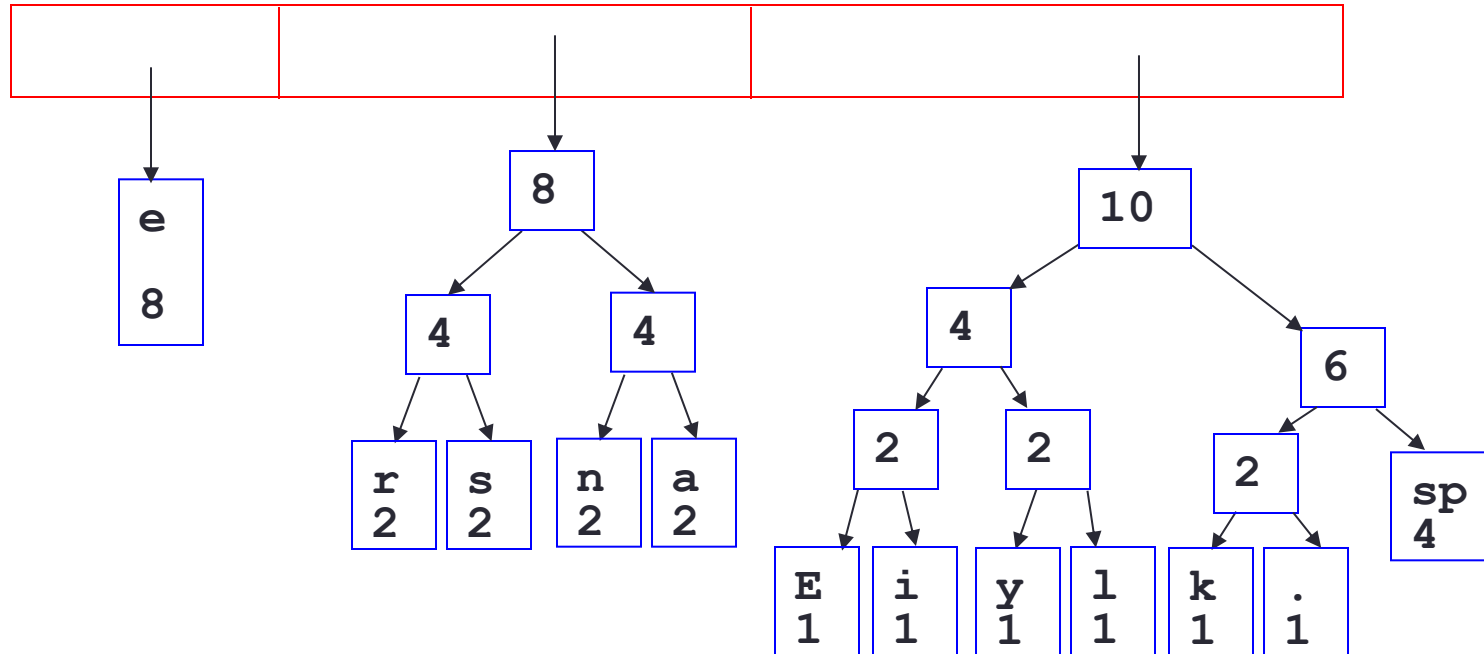
Building a Tree



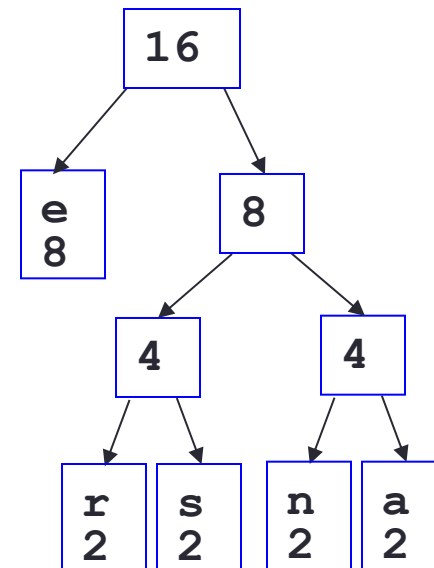
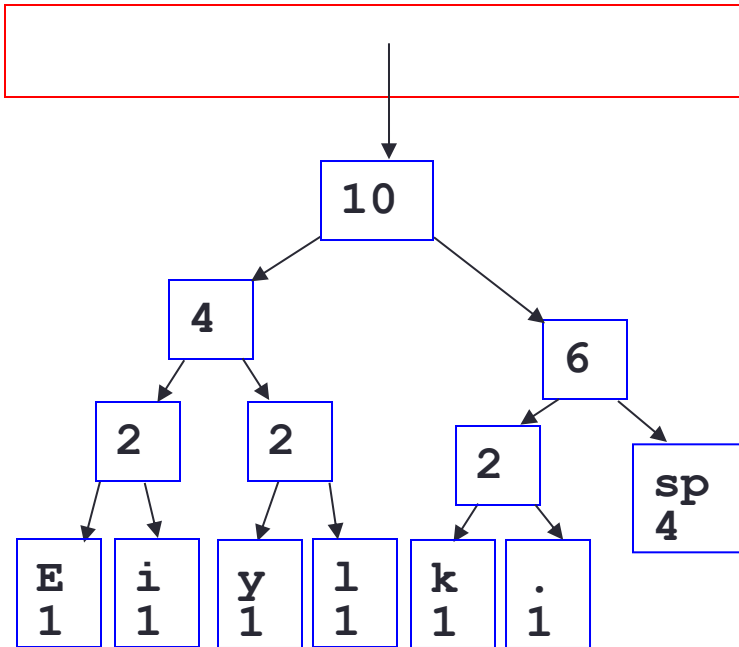
Building a Tree



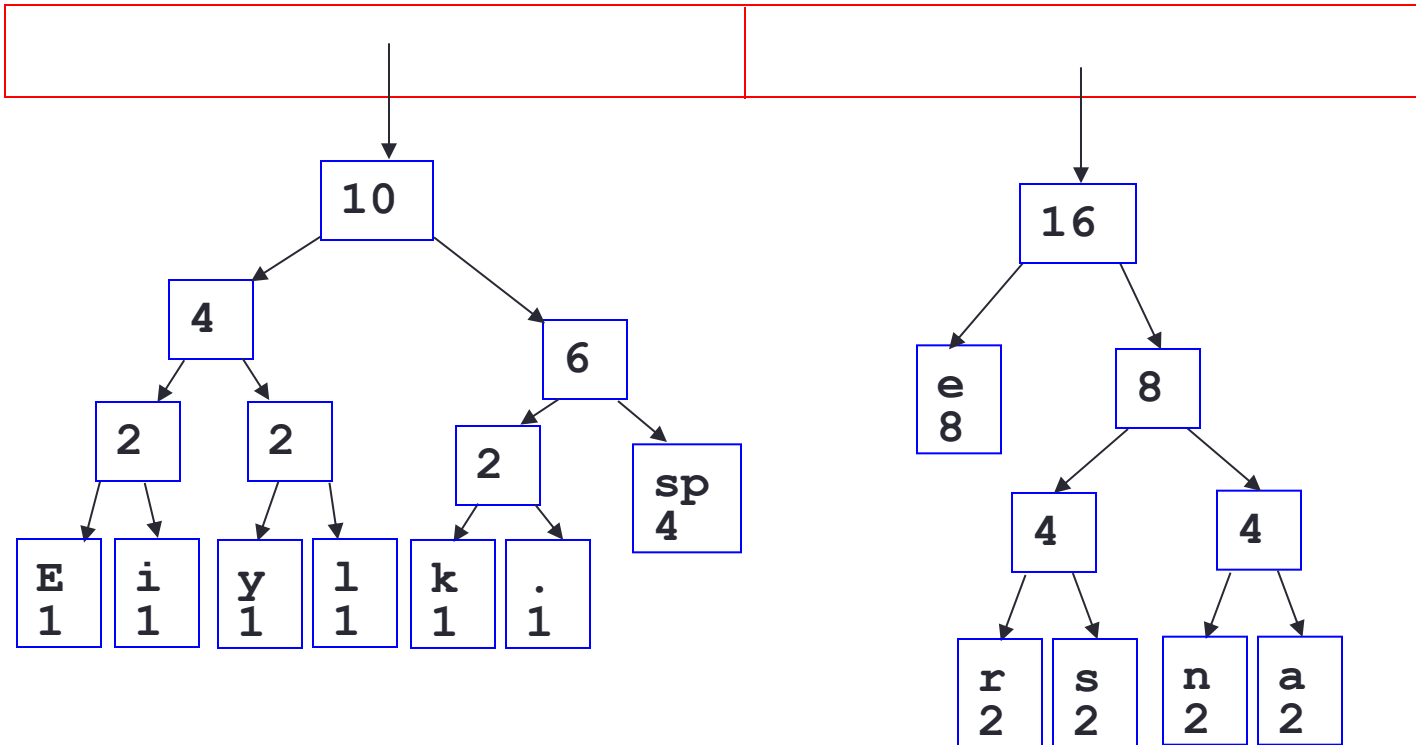
Building a Tree



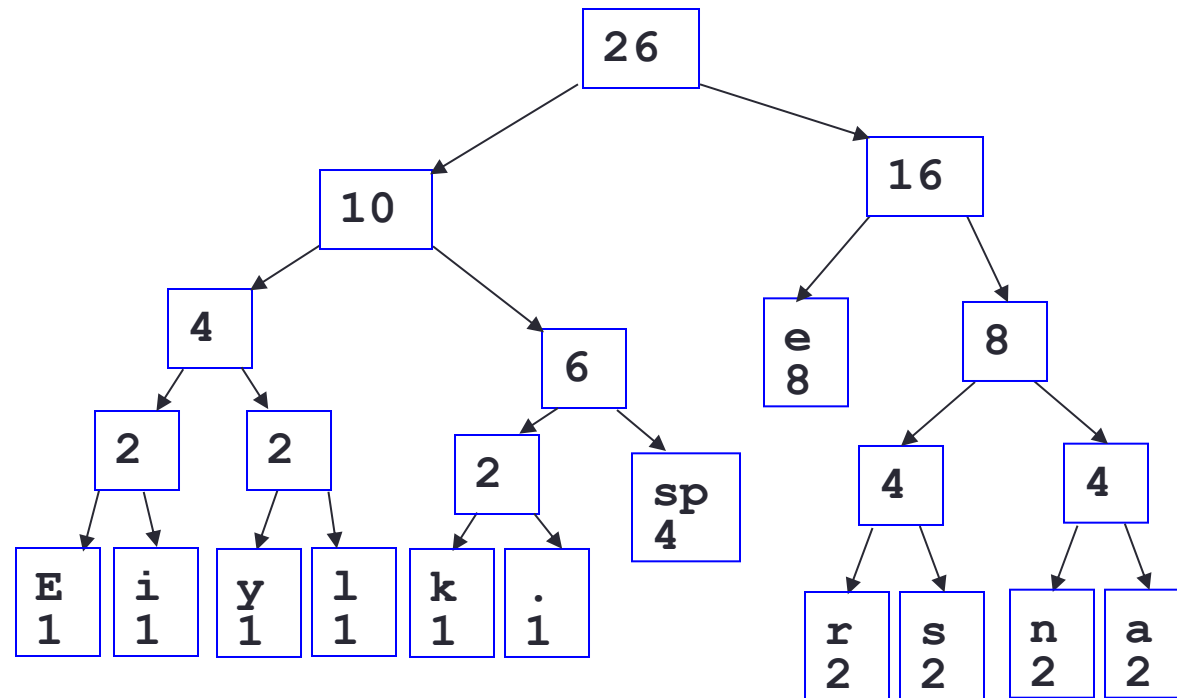
Building a Tree



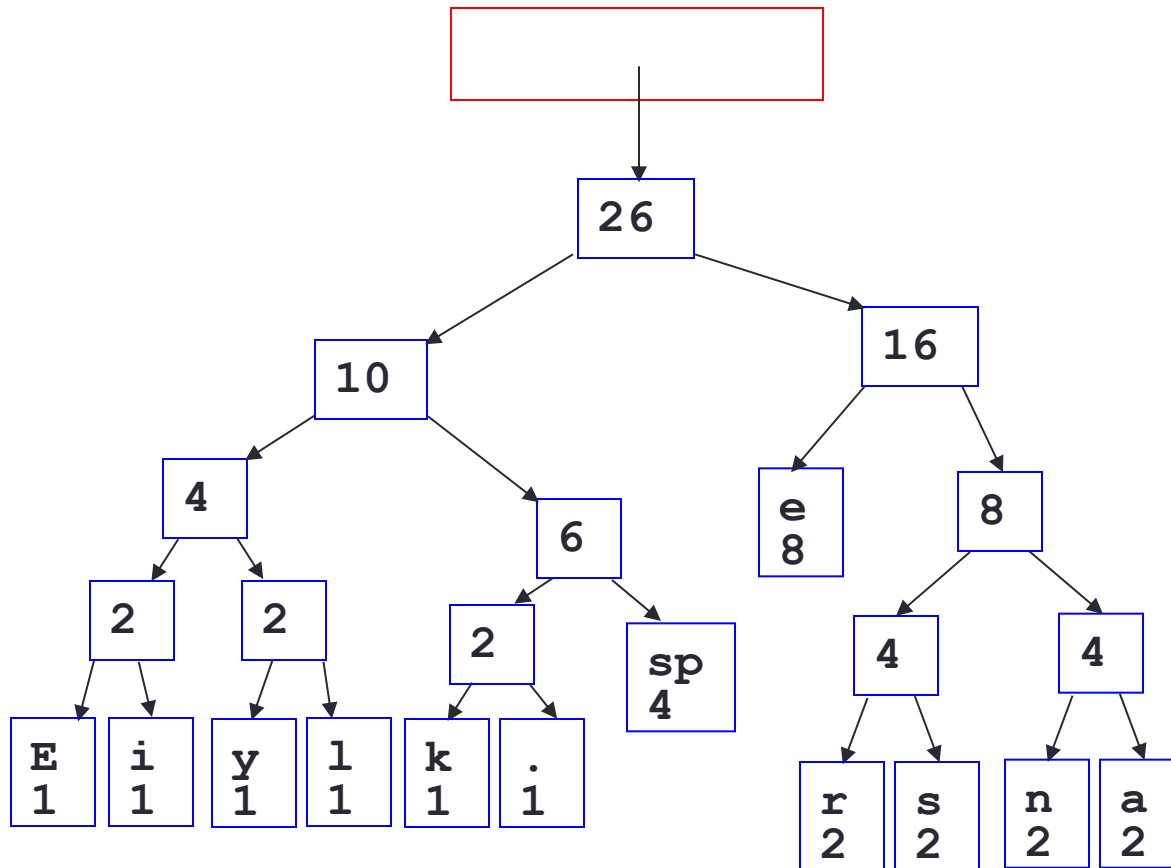
Building a Tree



Building a Tree

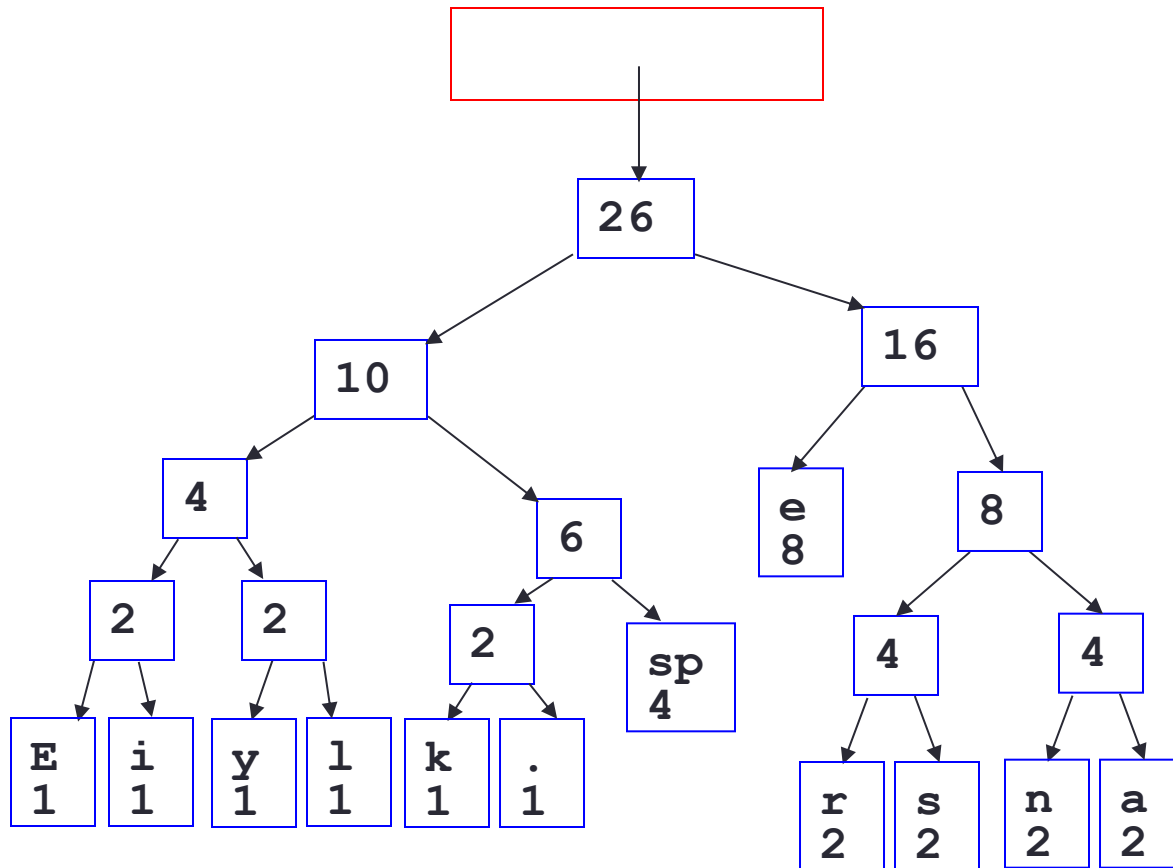


Building a Tree



• After enqueueing this node there is only one node left in priority queue.

Building a Tree

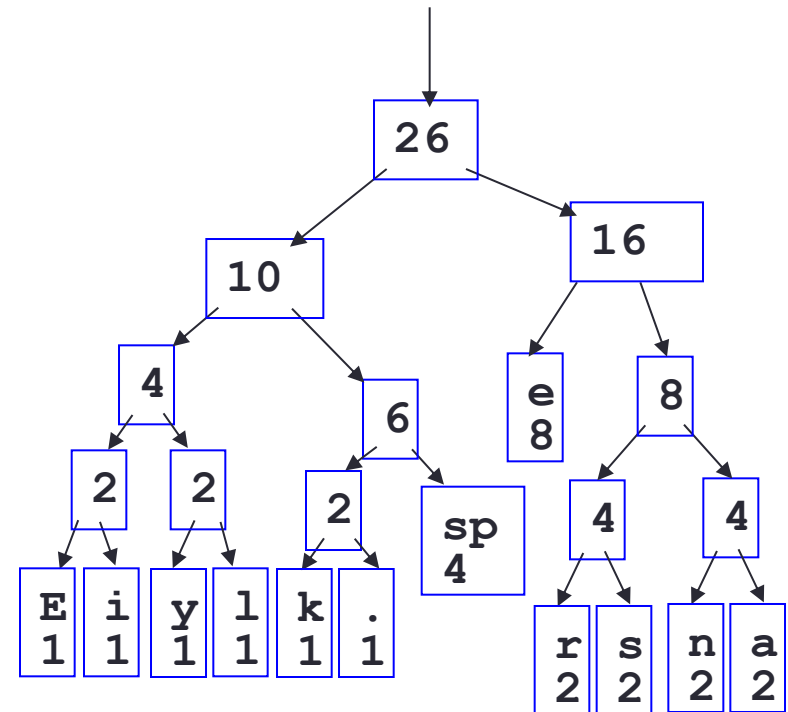


What happened to the characters with low frequencies?

Building a Tree

This tree contains the new code words for each character.

Frequency of root node should equal number of characters in text.

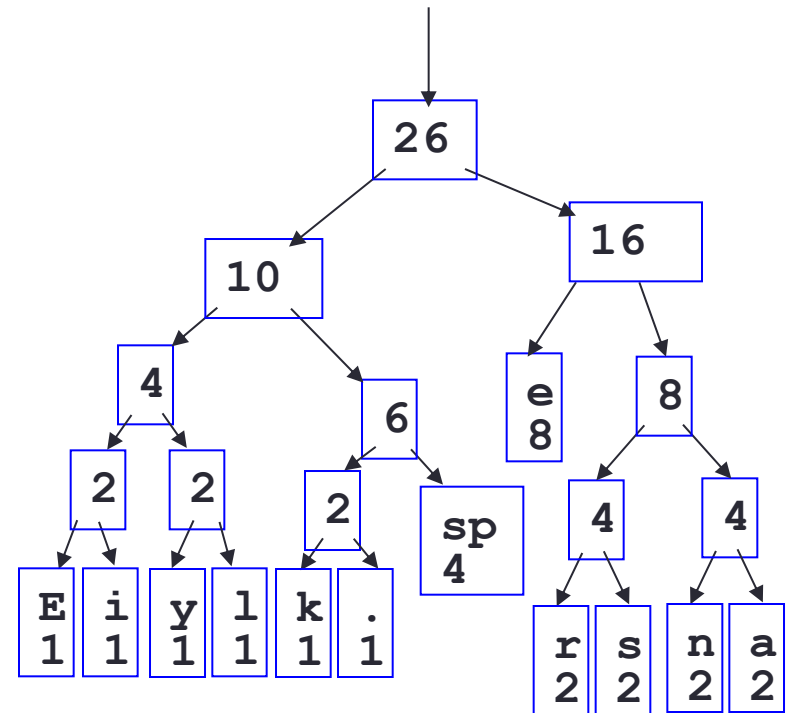


Eerie eyes seen near lake. □ 26 characters

Encoding the File

Traverse Tree for Codes

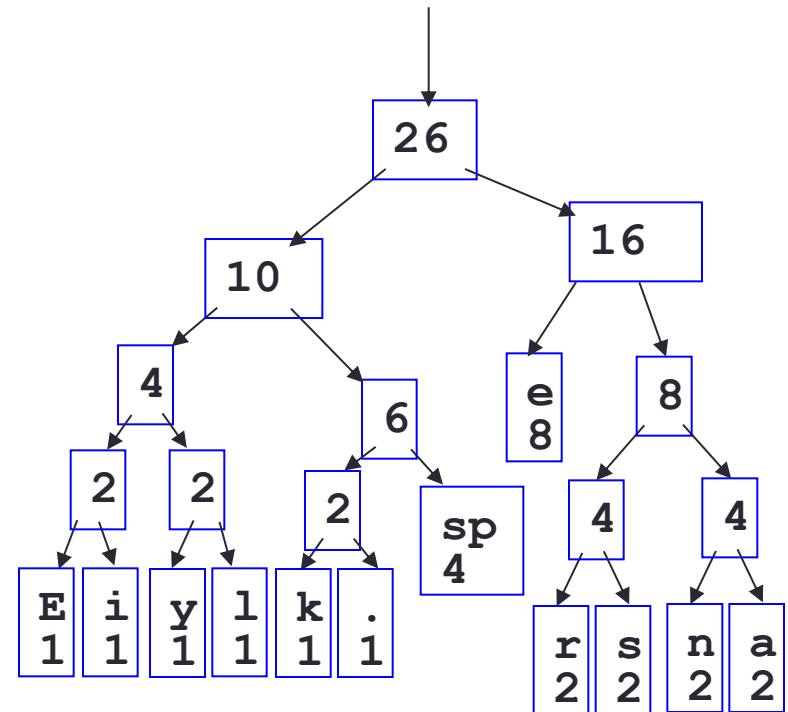
- Perform a traversal of the tree to obtain new code words
- Going left is a 0 going right is a 1
- code word is only completed when a leaf node is reached



Encoding the File

Traverse Tree for Codes

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111



Encoding the File

- Rescan text and encode file using new code words

Eerie eyes seen near lake.

```
0000101100000110011
1000101011011010011
1110101111110001100
1111110100100101
```

- No need for a separator character.

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Encoding the File Results

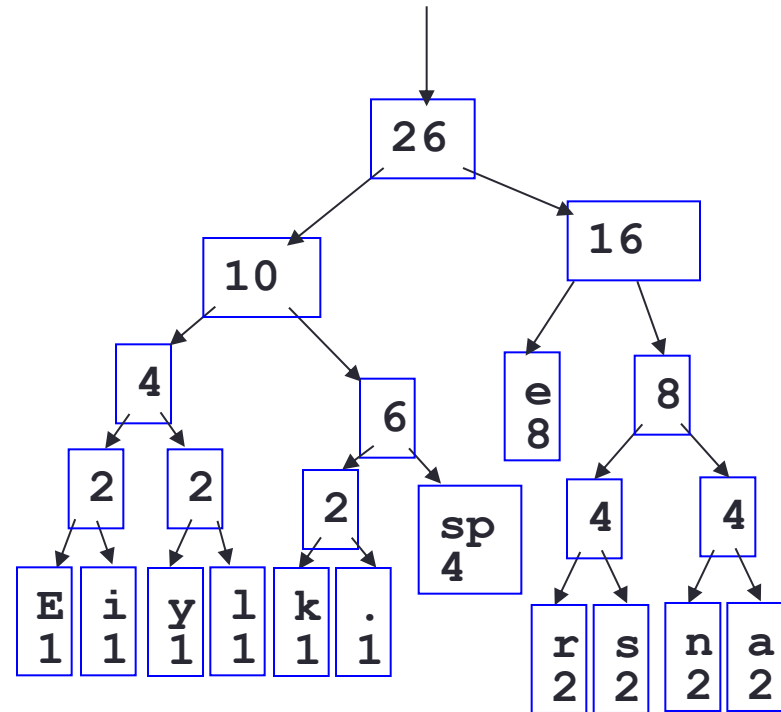
- Have we made things any better?
- 73 bits to encode the text
- ASCII would take $8 * 26 = 208$ bits
- How much percentage did we save?

```
0000101100000110011
1000101011011010011
1110101111110001100
1111110100100101
```

Decoding the File

- Once receiver has tree it scans incoming bit stream
- 0 $\Rightarrow$ go left
- 1 $\Rightarrow$ go right

101000110111101111
01111000100110101



Decoding the File

```
101000110111101111
01111000100110101
```

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

e

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

e

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

ee

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

ee

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

eel

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

eel

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

```
101000110111101111  
01111000100110101
```

eel<space>

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

```
101000110111101111  
01111000100110101
```

eel<space>

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

```
101000110111101111  
01111000100110101
```

eel<space>s

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

eel<space>s

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

eel<space>sn

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

eel<space>sn

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

eel<space>sna

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

```
101000110111101111
01111000100110101
```

eel<space>sna

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

```
101000110111101111
01111000100110101
```

eel<space>snai

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

eel<space>snai

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

```
101000110111101111
01111000100110101
```

eel<space>snail

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

eel<space>snail

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Decoding the File

101000110111101111
01111000100110101

eel<space>snail.

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Summary

- Huffman coding is a technique used to compress files for transmission
- Uses statistical coding
 - more frequently used symbols have shorter code words
- Works well for text and fax transmissions
- An application that uses several data structures